

UT3. Trabajar con Bases de datos (BBDD) en PHP

DESARROLLO WEB EN ENTORNO SERVIDOR (DWS)



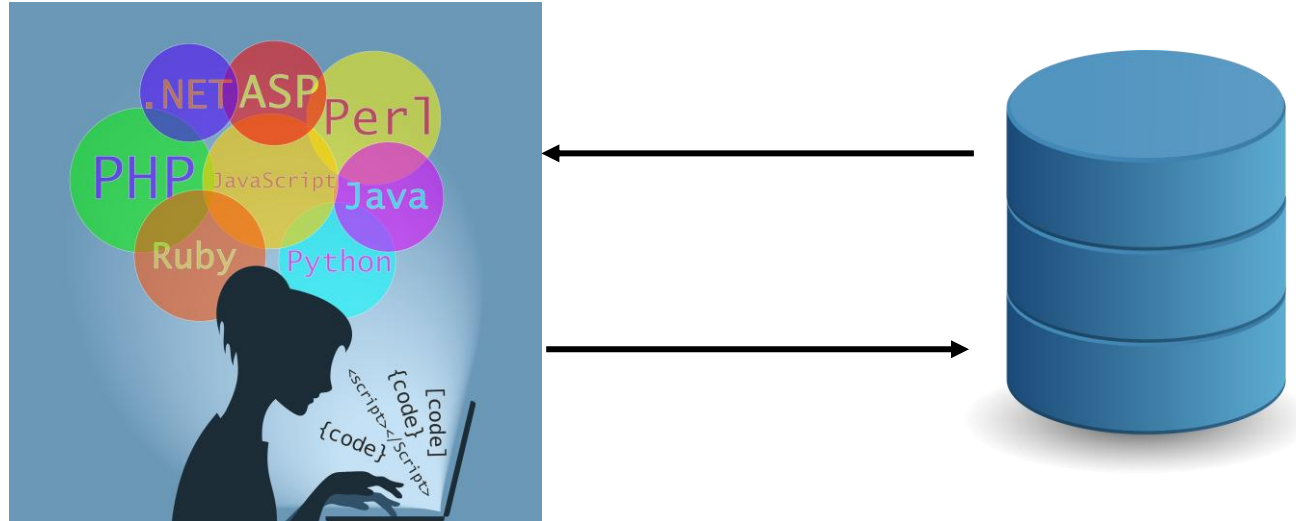
Contenidos

1. Acceso a Bases de datos desde PHP.
 1. MariaDB/MySQL.
2. Utilización de bases de datos MariaDB/MySQL en PHP.
 1. Extensión MySQLi.
 2. PHP Data Objects (PDO).
3. Errores y manejo de excepciones.

Acceso a base de datos desde PHP

UT03. TRABAJAR CON BASES DE DATOS (BBDD) EN PHP

Acceso a Bases de datos desde PHP. Introducción.



Acceso a Bases de datos desde PHP. Introducción.

En bases de datos existen diferentes criterios o maneras de clasificar las bases de datos:

- Según el modelo de datos:
 - Relacionales (SQL)
 - No relacionales (no-SQL)
- Según la ubicación:
 - Centralizadas
 - Distribuidas
 - En la nube
- Según la variabilidad de los datos:
 - Estáticas
 - Dinámicas
- Según el tiempo de vida:
 - Temporales
 - Permanentes



Acceso a Bases de datos desde PHP. Introducción.

Nosotros nos centraremos en **el modelo de datos**:

- Según el modelo de datos:
 - Relacionales (SQL): **MySQL**, PostgreSQL, Oracle, SQL Server, **MariaDB**...
 - No relacionales (no-SQL): MongoDB (documentos), Cassandra (columnas), Redis (clave-valor), Neo4j (grafos)...

Característica	Relacionales (SQL)	No Relacionales (no-SQL)
Modelo de datos	Usan tablas (filas-columnas)	Documentos, clave-valor, grafo, columnas
Esquema	Fijo, rígido	Flexible, sin esquema
Lenguaje de consulta	SQL	Varios (JSON, APIs, etc.)
Relaciones	Fuertes (joins, llaves)	Limitadas o manejadas por la app
Escalabilidad	Vertical (más potencia un servidor)	Horizontal (más servidores)
Consistencia	Fuerte	Generalmente eventual



Acceso a Bases de datos desde PHP. MariaDB/MySQL.

MariaDB: Es un SGBD relacional muy popular de código abierto (licencia GPL – 100% software libre) creado por los desarrolladores de MySQL en el año 2009 tras la compra de MySQL por parte de Oracle. Su soporte es llevado a cabo por una gran comunidad de usuarios MariaDB y presenta soporte empresarial a través de MariaDB Corporation.

MySQL: Es un SGBD relacional creado en 1995 por el grupo MySQL que fue finalmente comprado por Oracle en 2010. Actualmente tiene licencia GPL, pero algunas versiones avanzadas tienen licencia comercial y presenta soporte de Oracle más la comunidad de MySQL.

Ambos son 100% compatibles y son muy utilizados en lenguajes de programación como PHP, Java, Python, etc. Y son soportadas por las diferentes plataformas más comunes en la actualidad como Linux, Windows o macOS.

Comparten motores de almacenamiento como InnoDB



Utilización de bases de datos MariaDB/MySQL en PHP

UT03. TRABAJAR CON BASES DE DATOS (BBDD) EN PHP



Utilización de bases de datos MariaDB/MySQL en PHP.

Introducción

En PHP hay múltiples drivers para manejar los SGBD más extendidos.

También existe extensiones que proveen una capa de abstracción sobre las bases de datos, la más utilizada es **PHP Data Objects (PDO)**.

PDO permite manejar diferentes bases de datos con una interfaz común con lo que si se cambia de SGBD no hay que modificar el código.



Utilización de bases de datos MariaDB/MySQL en PHP.

Introducción - PDO

El primer paso para trabajar con una base de datos es obtener una conexión a la misma. Para ello se utilizará un objeto de la clase PDO.

```
public PDO::_construct(string $dsn[, string $username [, string $passwd [, array $options]]])
```

Donde:

\$dsn: Es una cadena que especifica que driver hay que usar, la localización y el nombre de la base de datos:

```
mysql:dbname=<baseDeDatos>;host=<ip o nombre>
```

\$username: es el nombre de usuario de acceso a la base de datos.

\$passwd: es la contraseña o clave del usuario para acceder a la base de datos.

\$options: es un array de opciones (por ejemplo: puede usarse en el caso de querer conexiones persistentes array(PDO::ATTR_PERSISTENT => true)).



Utilización de bases de datos MariaDB/MySQL en PHP.

Conexión a una base de datos - PDO

En caso de que no se pueda conectar a una base de datos, el constructor de **PDO** lanza una excepción **PDOException**.

Veamos un ejemplo (fichero **bd_connect.php**):

```
<?php
$cadena_conexion = 'mysql:dbname=empresa;host=127.0.0.1';
$usuario = 'root';
$clave = '';

try {
    $bd = new PDO($cadena_conexion, $usuario, $clave);
    $bd -> close();
} catch (PDOException $e) {
    echo 'Error con la base de datos: ' . $e->getMessage();
}
```



Utilización de bases de datos MariaDB/MySQL en PHP.

Recuperación y presentación de datos – PDO

Veremos el ejemplo de un fichero **bd_listar_usuario.php** que se muestra todos los datos de la tabla de *usuarios* de una base de datos denominada *empresa*:

Detalles del código ejemplo mostrado:

- **\$bd:** Recoge el objeto de la conexión realizada con PDO.
- **query(\$sql):** Ejecuta la consulta la cadena \$sql en el SGBD.
- **\$usuarios:** Recoge el resultado de la consulta \$sql realizada.
- **\$row:** Recoge cada una las t-uplas del resultado de la consulta realizada.



Utilización de bases de datos MariaDB/MySQL en PHP.

Inserción, borrado y actualización – PDO

Veremos el ejemplo de un fichero **bd_insert_update_delete.php** que realiza las operaciones indicadas de insertar, actualizar y borrar acorde con la consulta SQL a ser ejecutada:

Detalles del código ejemplo mostrado:

- **\$ins:** Recoge la sentencia SQL de inserción en la tabla.
- **\$upd:** Recoge la sentencia SQL de actualización en la tabla.
- **\$del:** Recoge la sentencia SQL de borrado en la tabla.



Utilización de bases de datos MariaDB/MySQL en PHP.

Transacciones – PDO

Una transacción consiste en un **conjunto de operaciones que deben realizarse de forma atómica**, es decir se realizan todas o ninguna.

Por ejemplo en una compra:

- **Operación 1:** Quitar el saldo al cliente que realiza la compra.
- **Operación 2:** Aumentar el saldo del vendedor que realiza la venta.



Si por cualquier motivo, el aumento del saldo produce un error, no se puede ejecutar la primera.

Es por ello que se indica que las dos operaciones forman una única **transacción**:

- Comienzo o inicio de una transacción: **beginTransaction()**.
- Se realiza un **commit()** cuando se desee finalizarla.
- Se realiza un **rollBack()** cuando se deshacen las operaciones realizadas desde el inicio.

Veremos el ejemplo **transaccion.php** para explicar en mayor detalle lo indicado.

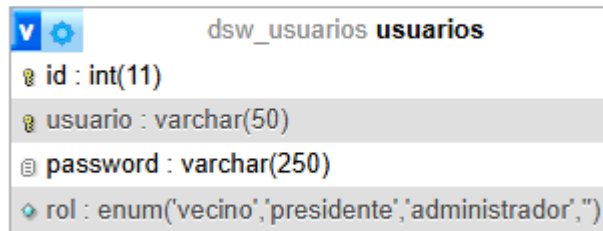


Utilización de bases de datos MariaDB/MySQL en PHP.

Ejemplo CRUD – PDO

Para desarrollar este sencillo ejemplo se tienen los siguientes datos:

- Base de datos: **dsw_usuarios**
- Tabla: **usuarios**
- Conexión a la base de datos: **PDO**



	dsw_usuarios usuarios
id	int(11)
usuario	varchar(50)
password	varchar(250)
rol	enum('vecino','presidente','administrador', '')

id: es la clave primaria

usuario: es **Unique**

password: no puede ser nulo

rol: puede ser vecino, presidente o administrador



Utilización de bases de datos MariaDB/MySQL en PHP.

Ejemplo CRUD – PDO

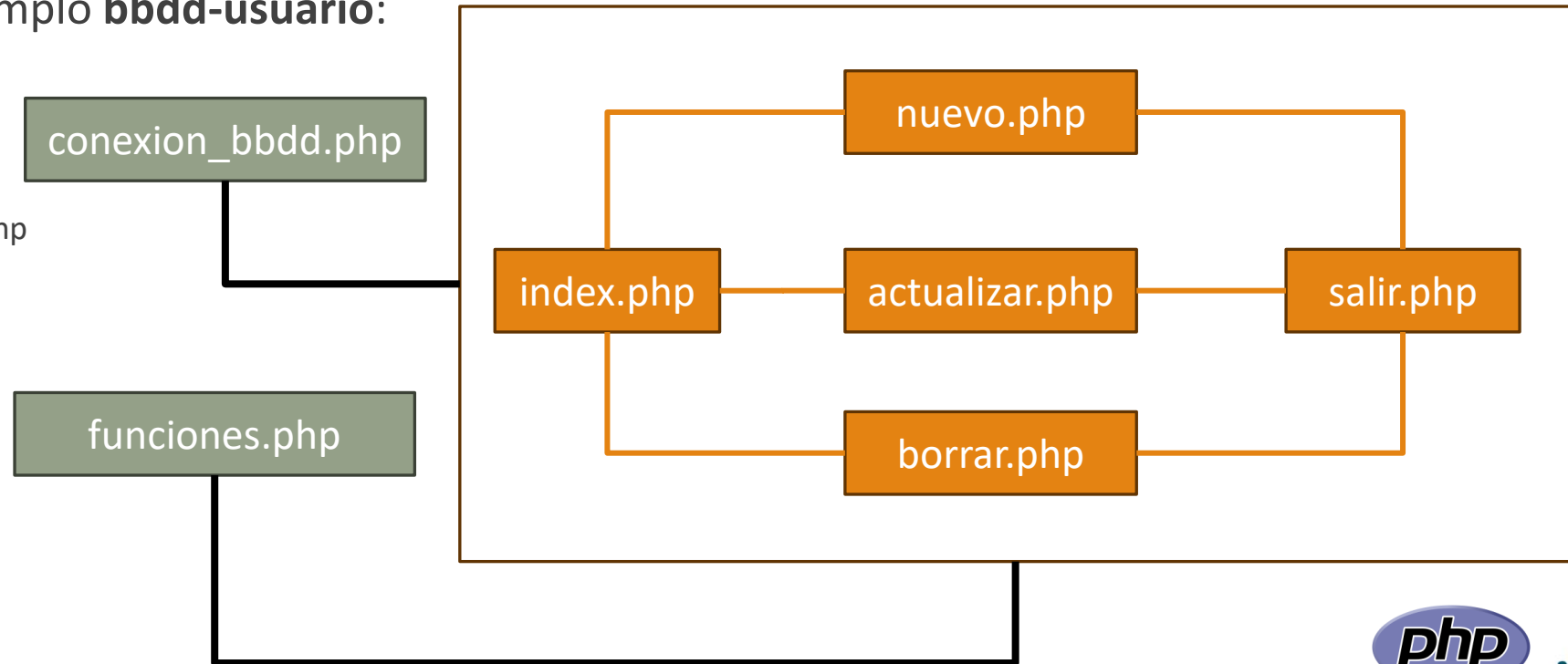
Estructura del ejemplo **bbdd-usuario**:

- code:

- actualizar.php
- borrar.php
- conexion_bbdd.php
- funciones.php
- index.php
- nuevo.php
- salir.php

- sql:

- usuarios.sql



Utilización de bases de datos MariaDB/MySQL en PHP.

Ejemplo CRUD – PDO

Estructura del ejemplo **bbdd-usuario**:

- code:
 - actualizar.php
 - borrar.php
 - conexión_bbdd.php
 - funciones.php
 - index.php
 - nuevo.php
 - salir.php
- sql:
 - usuarios.sql



Utilización de bases de datos MariaDB/MySQL en PHP.

Introducción – MySQLi

El primer paso para trabajar con una base de datos es obtener una conexión a la misma. Para ello, en esta ocasión se utilizará MySQLi.

MySQLi (MySQL improved extension – versión mejorada del antiguo mysql) es una extensión de PHP que permite conectarse y trabajar con bases de datos MySQL/MariaDB.

Las operaciones que MySQLi permite realizar son las siguientes:

- Conectarse con una base de datos MySQL/MariaDB.
- Ejecutar y Recuperar los resultados de las consultas SQL.
- Usar sentencias preparadas por mayor seguridad.
- Manejar errores y excepciones de conexión.
- Trabajar con objetos o de forma procedimental.



Utilización de bases de datos MariaDB/MySQL en PHP.

Conexión a una base de datos – MySQLi

Para conectarse a una base de datos MySQLi nos ofrece dos formas de trabajar, orientada a objetos (o POO) y la forma procedimental.

Veamos un ejemplo (fichero **bd_connect_mysqli.php**):

```
$servidor = "localhost";  
$usuario = "root";  
$contraseña = "";  
$base_datos = "test_db";
```

```
// Crear la conexión con Programación Orientada a Objetos (POO)  
$conn = new mysqli($servidor, $usuario, $contraseña, $base_datos);
```

```
// Comprobar si ha habido un error  
if ($conn->connect_error) {  
    die("Se ha producido un error en la conexión: " . $conn->connect_error);  
}
```

```
echo "Se ha conectado correctamente";  
$conn -> close();
```

```
$servidor = "localhost";  
$usuario = "root";  
$contraseña = "";  
$base_datos = "test_db";
```

```
// Crear la conexión de forma procedimental  
$conn = mysqli_connect("localhost", "root", "", "test_db");
```

```
// Comprobar si ha habido un error  
if (!$conn) {  
    die("Se ha producido un error en la conexión: " .  
        mysqli_connect_error());  
}  
echo "Se ha conectado correctamente";  
mysqli_close($conn);
```



Utilización de bases de datos MariaDB/MySQL en PHP.

Recuperación y presentación de datos – MySQLi

Veremos el ejemplo de un fichero **bd_listar_usuario_mysqli.php** que se muestra todos los datos de la tabla de *usuarios* de una base de datos denominada *empresa*:

Detalles del código ejemplo mostrado:

- Se muestra como es la consulta trabajando de forma procedimental.
- Se muestra como es la consulta trabajando POO.



Utilización de bases de datos MariaDB/MySQL en PHP.

Inserción, borrado y actualización – MySQLi

Veremos el ejemplo de un fichero **bd_insert_update_delete_mysqli.php** que realiza las operaciones indicadas de insertar, actualizar y borrar acorde con la consulta SQL a ser ejecutada:

Detalles del código ejemplo mostrado:

- **\$ins:** Recoge la sentencia SQL de inserción en la tabla.
- **\$upd:** Recoge la sentencia SQL de actualización en la tabla.
- **\$del:** Recoge la sentencia SQL de borrado en la tabla.



Detalles del código ejemplo mostrado:

- Se muestra como es la consulta trabajando de forma procedimental.
- Se muestra como es la consulta trabajando POO.



Utilización de bases de datos MariaDB/MySQL en PHP.

Preparación de consultas – PDO/MySQLi

Una consulta SQL a ser realizada se puede expresar tal como se ha visto en los ejemplos vistos previamente o se pueden realizar de forma segura evitando inyecciones SQL.

En el fichero **prepared_statement** se muestra un ejemplo de cómo se debe realizar las consultas SQL de forma segura.

Se puede aplicar tanto para:

- Conexiones con PDO (**prepared_statement_pdo.php**).
- Conexiones con MySQLi (**prepared_statement_mysqli.php**).



Es por ello que se deben realizar todas las consultas utilizando esta forma en vez de la vista en los ejemplos anteriores.

Para más información consultar <https://www.php.net/manual/en/mysqli-stmt.bind-param.php>

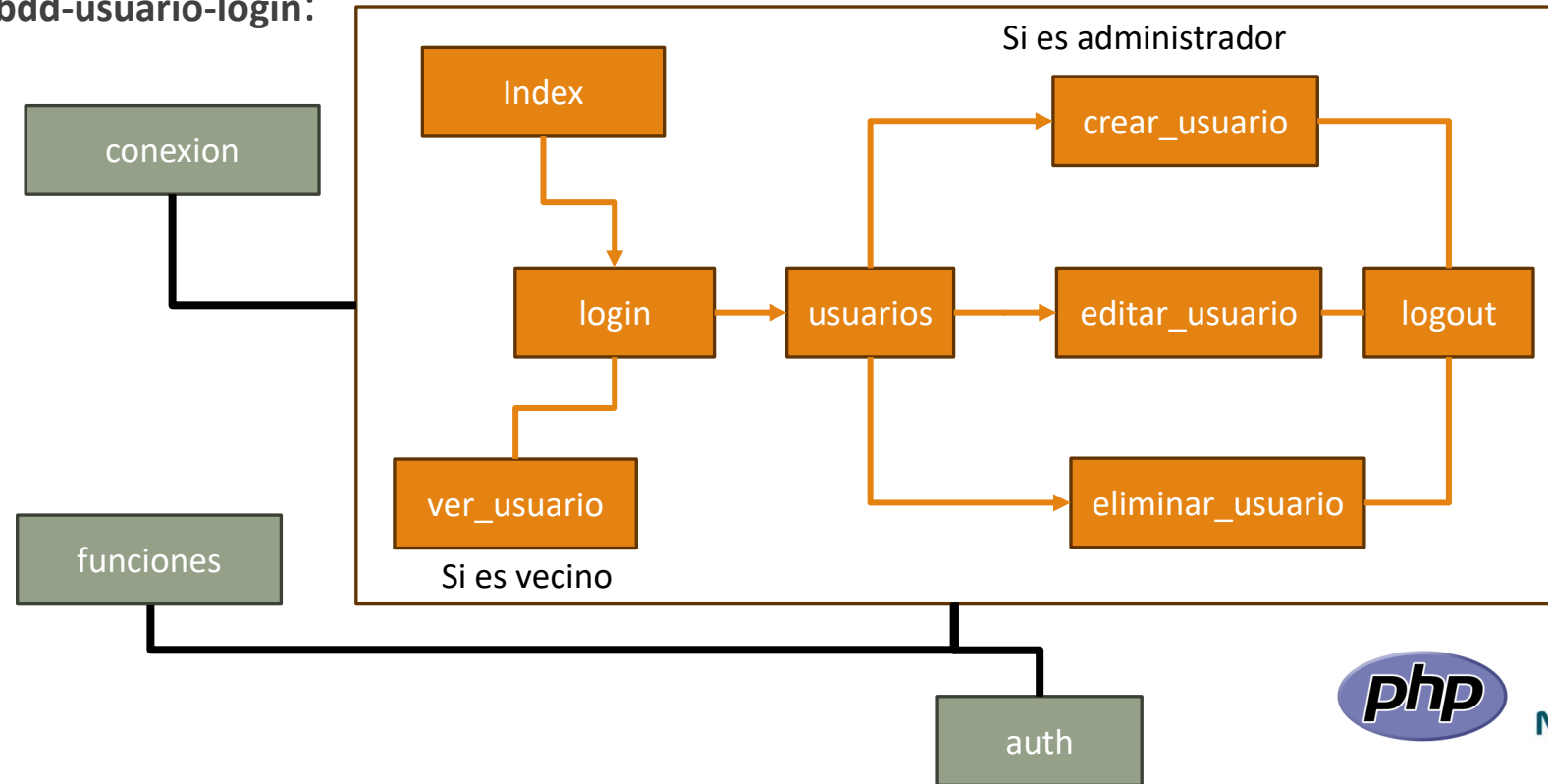


Utilización de bases de datos MariaDB/MySQL en PHP.

Ejemplo CRUD – MySQLi

Estructura del ejemplo **bbdd-usuario-login**:

- code:
 - auth.php
 - conexion.php
 - crear_usuario.php
 - editar_usuario.php
 - eliminar_usuario.php
 - funciones.php
 - index.php
 - login.php
 - logout.php
 - usuarios.php
 - ver_usuario.php
- sql:
 - usuarios.sql



Utilización de bases de datos MariaDB/MySQL en PHP.

Resumen – PDO/MySQLi

Manejo de errores:

- MySQLi: Errores se manejan con `mysqli_error()`.
- PDO: Soporta excepciones (try/catch).

Ventajas:

PDO	MySQLi
Compatible con muchos SGBD. Más seguro en el manejo de excepciones. Código limpio con menos sentencias. Facilidad para migrar entre motores SGBD. Orientado a Objetos.	Ligero y rápido sólo para MySQL/MariaDB. Soporta estilos POO y procedimental. Alto rendimiento en BBDD MySQL/MariaDB de gran tamaño.



Utilización de bases de datos MariaDB/MySQL en PHP.

Errores y manejo de excepciones

El sistema de control de errores de PHP ha ido evolucionando a lo largo de las diferentes versiones.

En el sistema básico, se generan errores de diferentes tipos:

- En versiones PHP 5 en adelante, sistemas de excepciones similares a Java con *try/catch/finally*.
- En PHP 7 en adelante, las excepciones de la clase *Error*.

Como errores típicos se tienen (configurados en **php.ini**):

- `error_reporting`: indica qué errores deben reportarse. Lo normal es utilizar `E_ALL`.
- `display_error`: señala si los mensajes de error deben aparecer en la salida del script.
- `log_error`: indica si los mensajes de error deben almacenarse en un fichero.
- `error_log`: si la directiva anterior está activada, es la ruta en la que se guardan los mensajes de error.



Utilización de bases de datos MariaDB/MySQL en PHP.

Errores y manejo de excepciones

La función **error_reporting()** permite cambiar el valor de la directiva vista **error_reporting** en tiempo de ejecución.

Es posible definir una función propia para que se encargue de los errores utilizando la función **set_error_handler()**. Por ejemplo:

```
function manejadorErrores($errno, $str, $file, $line){  
    echo "Ocurrió el error: $errno";  
}  
  
set_error_handler("manejadorErrores");  
  
$a = $b;                // causa error, $b no está inicializada
```



Utilización de bases de datos MariaDB/MySQL en PHP.

Errores y manejo de excepciones

Otra opción para indicar un error es lanzar una excepción. Para controlar las mismas se usan el bloque *try/catch/finally*, tal como haríamos en *Java*.

Cuanto se lanza una excepción es capturada por el bloque *catch* y se ejecutan las sentencias que engloban dicho *try/catch*.

Se verá en el ejemplo **excepciones.php** el uso de la función que recibe dos argumentos y lanza una excepción si el segundo tiene el valor de 0. La excepción es lanzada mediante la palabra reservada *throw*, que recibe como argumento un objeto de la clase *Exception*.

